OFFICIAL COURSE PROPOSAL • Zero-to-Advanced Computer Science Track

Problem Solving & Algorithmic Foundations

A structured, 120-hour intensive program leveraging world-class ICPC sheets to build interview-ready software engineers and competitive programming champions.

TOTAL SCOPE
120 Hours / 40 Sessions

TARGET PACE
3 Sessions / Week (3 hrs)

PREREQUISITES
Level 0 (Zero Barrier)

EXECUTIVE SUMMARY

Why Offer This Course at Your Institution?

Beyond Syntax-Only Coding

Replaces conventional syntax teaching with algorithmic problem-solving logic and computational thinking.

Top-Tier Tech Interview Mastery

Directly aligns with FAANG & Tier-1 technical screening standards, focusing on optimal space/time complexity.

Competitive Programming Growth

Prepares students for local and regional ICPC/ECPC contests through rigorous Codeforces sheet integration.

CORE METHODOLOGY

The Three Pillars of Technical Excellence

PILLAR 01

Syntax & Mechanics

Standard I/O, fast execution, pointers, reference passing, and basic control structures.

PILLAR 02

Data Structures

Vectors, frequency arrays, sets, maps, stacks, queues, DSU, and custom comparators.

PILLAR 03

Algorithmic Depth

Binary search on answers, dynamic programming, graph traversals, and number theory.

CURRICULUM INTEGRATION

Dual-Sheet Practice Engine

ICPC Assiut University Sheets

Structured, topic-by-topic progression covering essential language syntax, loops, arrays, strings, standard math, and fundamental recursion.

Target: Complete Fundamental Mastery

Dr. Mostafa Saad Roadmap

Hand-picked Codeforces problems (A/B/C levels) focusing on pattern recognition, ad-hoc thinking, search-space reduction, and contest speed.

Target: High-Efficiency Problem Solving

TARGET AUDIENCE

Enrollment Profile & Zero-Barrier Entry

Designed For

- **CS Undergraduates:** Seeking rock-solid core CS fundamentals.
- **Aspiring Competitors:** Aiming for ICPC / ECPC regional ranking.
- **Job Seekers:** Preparing for tier-1 software engineering interviews.

Zero Entry Barrier

No prior coding background is required. The track starts strictly at Level 0, building logical thinking and syntax mechanics before introducing complex algorithms.

- ✓ Basic computer operation skills
- ✓ Commitment to 10+ hrs/week sheet solving

PROGRAM OUTCOMES

Measurable Student Growth Indicators

300+

Curated Problems

Verified on Codeforces judge

100%

Interview Readiness

Big-O analysis & optimization

O(N)

Optimal Mindset

Strict 10^8 ops TLE prevention

MODULE 01 (SESSIONS 1 - 6)

Level 0 Programming Onboarding & Complexity

18 Hours

SESSION	CORE TOPIC	PRACTICAL FOCUS & LAB EXERCISES
Session 01	**Programming Fundamentals I**	Variables, Data Types, Standard I/O, Fast I/O mechanics.
Session 02	**Programming Fundamentals II**	Conditional Branching (if-else, switch), relational logic.
Session 03	**Loops & Iteration Mechanics**	For, while, nested loops execution analysis.
Session 04	**Arrays & Strings Deep Dive**	1D & 2D array memory layout, ASCII operations.
Session 05	**Functions & Recursion Intro**	Scope, pass by value vs reference, base cases.
Session 06	**Big-O & Complexity Analysis**	Calculating operations, TLE prevention (10^8 ops rule).

MODULE 02 (SESSIONS 7 - 13)

OOP, STL & Array Optimization Techniques

21 Hours

SESSION	CORE TOPIC	PRACTICAL FOCUS
Session 07	**OOP & Custom Comparators**	Structs/Classes, member functions, custom sorting rules.
Session 08	**STL Containers & Iterators (1)**	Dynamic arrays (vector), pair, built-in algorithms.
Session 09	**STL Containers (Part 2)**	Set, map, priority_queue, stack, queue, deque.
Session 10	**Frequency Array Technique**	Direct address tables, O(1) element frequency counting.
Session 11	**Prefix Sum (1D & 2D)**	Range sum queries in O(1) time complexity.
Session 12	**Partial Sum / Difference Array**	Dynamic range updates in O(1) complexity.
Session 13	**Coordinate Compression**	Mapping sparse large ranges (10^9) to dense indices (10^5).

MODULE 03 (SESSIONS 14 - 20)

Subarray Math, Binary Search & Backtracking

21 Hours

SESSION	CORE TOPIC	PRACTICAL FOCUS
Session 14	**Contribution Technique**	Calculating element contributions across continuous subarrays.
Session 15	**Binary Search Mechanics**	Lower bound, upper bound from scratch.
Session 16	**Binary Search on Answer**	Monotonic functions, threshold search.
Session 17	**Interactive Problems**	Flushing output buffers, query-response strategy.
Session 18	**Recursion & State Trees**	Call stack analysis, power set generation.
Session 19	**Backtracking Fundamentals**	Combinations, permutations under constraints.
Session 20	**Backtracking & Pruning**	Pruning search spaces, N-Queens problem, Sudoku solver.

MODULE 04 (SESSIONS 21 - 27)

Bitwise Operations, Number Theory & Combinatorics

21 Hours

SESSION	CORE TOPIC	PRACTICAL FOCUS
Session 21	**Bitmasks Fundamentals**	Bitwise operators, setting/clearing bits, subset masking.
Session 22	**Bitset Optimizations**	Using std::bitset for N/64 speedup optimizations.
Session 23	**Number Theory (Part 1)**	GCD/LCM, Primality testing O(√N), Sieve of Eratosthenes.
Session 24	**Number Theory (Part 2)**	Prime Factorization, Segmented Sieve, Euler's Totient.
Session 25	**Modular Arithmetic & Power**	Binary Exponentiation O(log N), modular inverse.
Session 26	**Combinatorics (Part 1)**	nCr & nPr formulas, Pascal's Triangle pre-computation.
Session 27	**Combinatorics (Part 2)**	Inclusion-Exclusion Principle, Stars and Bars method.

MODULE 05 (SESSIONS 28 - 33)

Graph Theory, Traversals & Shortest Paths

18 Hours

SESSION	CORE TOPIC	PRACTICAL FOCUS
Session 28	**Graph Theory Intro**	Adjacency List vs Matrix, Directed/Undirected representations.
Session 29	**Depth-First Search (DFS)**	DFS recursion, connected components, cycle detection.
Session 30	**Breadth-First Search (BFS)**	Unweighted shortest paths, 2D grid pathfinding.
Session 31	**Dijkstra's Algorithm**	Priority queue relaxation, weighted shortest paths.
Session 32	**Disjoint Set Union (DSU)**	Path compression & union by rank for dynamic connectivity.
Session 33	**Minimum Spanning Tree**	Kruskal's Algorithm using DSU, Prim's Algorithm.

MODULE 06 (SESSIONS 34 - 40)

Dynamic Programming, String Hashing & Contests

21 Hours

SESSION	CORE TOPIC	PRACTICAL FOCUS
Session 34	**Dynamic Programming Intro**	Memoization (Top-Down DP), state representation.
Session 35	**Iterative DP & Tabulation**	Tabulation (Bottom-Up DP), space optimization.
Session 36	**Classic DP Patterns**	0/1 Knapsack, Coin Change, Longest Common Subsequence.
Session 37	**String Hashing Mechanics**	Polynomial rolling hash, O(1) substring comparison.
Session 38	**Permutation Hashing**	Hashing unordered sets & permutations for rapid checks.
Session 39	**Comprehensive ICPC Contest**	5-hour simulated team competition with 10+ multi-topic problems.
Session 40	**Editorial & Final Roadmap**	Editorial defense, code portfolio review, certification issuance.

TEACHING & EVALUATION

Session Architecture & Assessment Grading

3-Hour Session Architecture

First 45 Mins
Sheet review & problem editorial discussion

Next 75 Mins
Interactive topic lecture & proof analysis

Final 60 Mins
Live coding lab with immediate submission

Grading Weight Distribution

Weekly Sheet Progress (Codeforces)	40%
Mid-Track Speed Contest (3 hrs)	20%
Final ICPC Mock Competition (5 hrs)	30%
Code Review & Lab Participation	10%



Ready to Launch

Equipping your institution's students with world-class problem-solving confidence, top-tier software interview skills, and competitive excellence.

40 Sessions

120 Program Hours